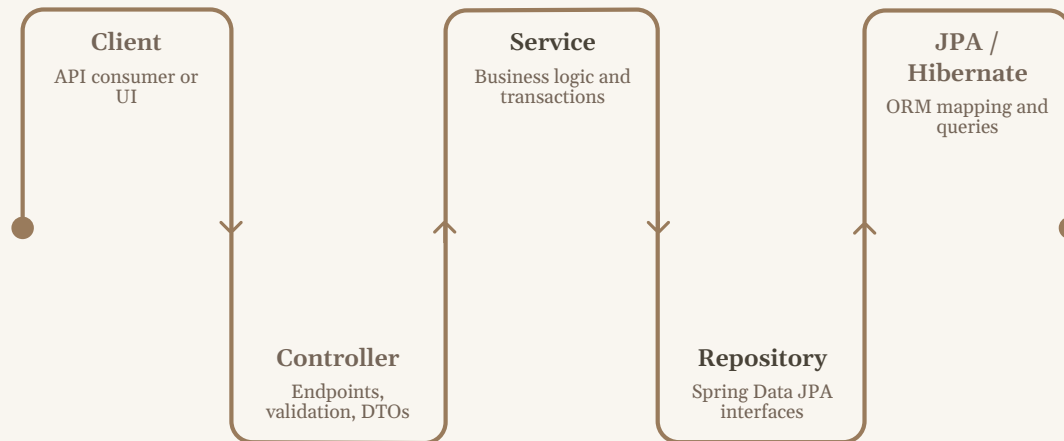
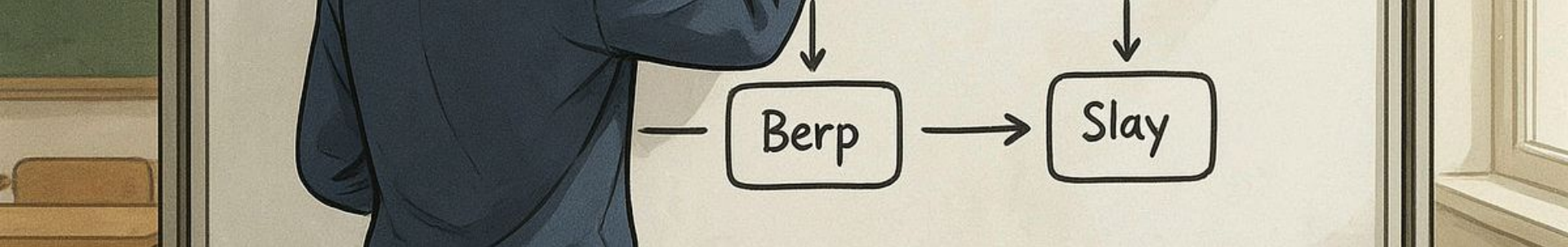




Data Management with Spring Boot

How a Spring Boot REST API persists data in MySQL using Spring Data JPA — covering entities, repositories, DTOs, validation, transactions, and pagination. Built around our **Student Management API**.





Where Data Fits in Our Application

Every layer in the Student Management API has a distinct, well-defined responsibility. Understanding the separation keeps code clean and maintainable.

Controller

Handles HTTP requests and responses. Translates between the web world and your service layer.

Service

Owns the business logic. Orchestrates operations like registration, enrollment, and validation rules.

Repository

Manages database access. Reads and writes data without you writing raw SQL.

Entity

Represents a database table as a Java class. One entity instance = one database row.

Defining an Entity

An **Entity** is a Java class mapped directly to a database table. Spring Data JPA uses annotations to tell Hibernate exactly how to create and manage the table.

```
@Entity
public class Student {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
    private String email;

}
```

→ **@Entity**
Marks this class as a JPA entity — a database table will be created for it.

→ **@GeneratedValue**
Tells the database to auto-generate the ID value on insert.

→ **@Id**
Declares the primary key of the table.

→ **Fields**
Each field (name, email) automatically becomes a column in the `student` table.

📄 Resulting table: student with columns id | name | email

The Repository Layer

A **repository** is an interface that gives you full database access without writing a single SQL statement. Just extend `JpaRepository` and you're done.

```
public interface StudentRepository
    extends JpaRepository<Student, Long> {

    // Nothing else needed!
}
```



`save()`

Insert or update a student record.



`findAll()`

Retrieve every student from the database.



`findById()`

Look up a single student by their ID.



`deleteById()`

Remove a student by their ID.



`count()`

Return the total number of student records.

✔ We write less SQL. Spring Data JPA generates the queries for us automatically.

CRUD Flow in Action

Let's trace exactly what happens when the client creates or retrieves a student — from the HTTP request all the way down to MySQL and back.

Creating a Student — POST /students



Fetching All Students — GET /students



The **Controller** never talks to the database directly. The **Service** contains the logic. The **Repository** is the only layer that touches MySQL. This clean separation makes the code easy to test and maintain.



Relationships Between Entities

Real-world data is rarely flat. JPA supports four types of relationships to model how entities connect — mirroring actual foreign key constraints in MySQL.

1

One-to-One

One entity links to exactly one other. Example: one student has one student profile.

2

One-to-Many

One entity links to many others. Example: one course has many students enrolled.

3

Many-to-One

Many entities link to one. Example: many students belong to one course. Most common in Student Management.

4

Many-to-Many

Many entities on both sides. Example: students can enroll in multiple courses; courses have many students.

📌 **Our focus today:** @ManyToOne — many students belong to one course. This is the most practical starting point.

Why Use a DTO?

Exposing your database entity directly from the API is a common mistake. It leaks internal fields, breaks API contracts, and creates security risks. A **Data Transfer Object (DTO)** solves this cleanly.

✗ Without DTO

```
// Entity exposed directly
StudentEntity {
  id
  name
  email
  passwordHash ← exposed!
  internalNotes ← exposed!
  auditTimestamps ← exposed!
}
```

✓ With DTO

```
// Only what the API needs
StudentResponseDTO {
  id
  name
  email
  // nothing else
}
```



Security

Sensitive or internal fields never reach the client.



Stable API Contract

Internal entity changes don't break the public API.



Control

You decide exactly what fields the response includes.

Input Validation

Never trust data coming from the client. Spring Boot's `@Valid` and Bean Validation annotations automatically reject bad input before it reaches your service layer.

Request DTO with Validation

```
public class StudentRequest {  
  
    @NotBlank  
    private String name;  
  
    @Email  
    @NotBlank  
    private String email;  
  
}
```

Controller

```
@PostMapping("/students")  
public Student create(  
    @Valid @RequestBody  
    StudentRequest request) {  
    ...  
}
```

Key Annotations

`@NotBlank`

Rejects null or empty strings. Name cannot be blank.

`@Email`

Ensures the value matches a valid email format.

`@Valid`

Activates validation on the incoming request body.



Bad input like `{"name": "", "email": "abc"}` returns **HTTP 400 Bad Request** automatically — no extra code needed.

Transactions — All or Nothing

A **transaction** groups multiple database operations into one atomic unit. Either every operation succeeds together, or none of them are saved — the database stays consistent no matter what.

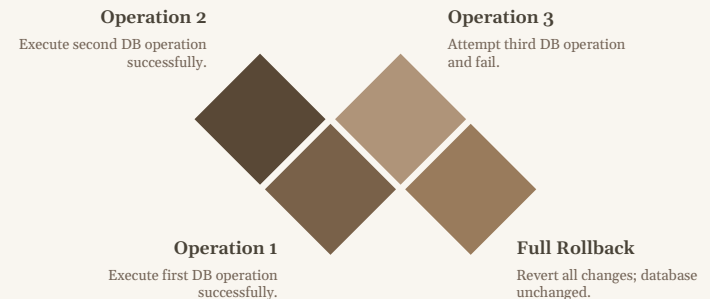
Example Code

```
@Transactional
public void registerStudent(...) {

    saveStudent();    // Step 1
    assignCourse();   // Step 2
    createAuditRecord(); // Step 3

}
```

What Happens on Failure



All Operations Succeed ✓

The transaction commits. All changes are permanently saved to MySQL.

Any Operation Fails ☒

The transaction rolls back. No partial data is written — the database is unchanged.

Pagination, Sorting & Filtering

Returning all records at once is a performance disaster at scale. Spring Data JPA makes it trivial to return data in manageable pages and in any sort order.

Example Requests

```
GET /students?page=0&size=10
```

```
GET /students?sort=name
```

```
GET /students?name=Rahul
```

Repository Method

```
Page<Student> findAll(  
    Pageable pageable);
```

Why This Matters

Without Pagination

100,000 records returned at once → slow response, high memory use, poor UX.

With Pagination

10 records per page → fast response, efficient queries, smooth user experience.

Spring's `Pageable` interface handles `page`, `size`, and `sort` parameters automatically. Just pass it into your repository method and Spring does the rest.

Advanced Topics — Learn Later

These topics are real challenges you'll encounter as the application grows. Don't worry about implementing them today — just know they exist so you can look them up at the right time.

1

Optimistic Locking

Prevents two users from overwriting each other's changes simultaneously. Uses a `@Version` field to detect conflicts and throw an exception instead of silently losing data.

2

N+1 Query Problem

Relationships can trigger one extra SQL query per record. Fetching 100 students with their courses fires 101 queries. Solved with `JOIN FETCH` or `@EntityGraph`.

3

JPQL & Custom Queries

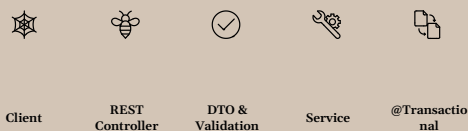
When derived method names aren't enough, write your own queries using JPQL (Java Persistence Query Language) or native SQL with `@Query`.

STUDY THESE AFTER TODAY'S HANDS-ON SESSION

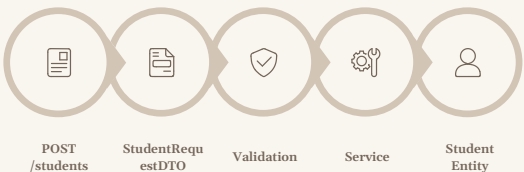
The Complete Data Flow

Here's the full picture — every layer working together from the client's HTTP request down to MySQL and back as a JSON response.

Layer Architecture



POST /students — Full Example



Entity

Database representation



Repository

Database access



DTO

API contract



Validation

Protect input



Transaction

Maintain consistency



Pagination

Handle large data